

SQL Tutorial

From Messy Tables to Clean Joins

1. The Original Table (The Problem)

Imagine a small training institute keeps ALL its information in one single table called Student_Records.

Look carefully at the data below. Something is clearly wrong.

Table: Student_Records

RollNo	StudentName	City	Phone	CourseName	Fees	Trainer	Trainer_Email
101	Aarav Sharma	Jaipur	98110-11223	SQL, Python	5000, 7000	Ravi Verma	ravi@abc.com
101	Aarav Sharma	Jaipur	98110-11223	Excel	3000	Neha Gupta	neha@abc.com
102	Diya Mehta	Delhi	99880-22334	SQL	5000	Ravi Verma	ravi@abc.com
103	Kabir Singh	Jaipur	97770-33445	Python, Excel	7000, 3000	Neha Gupta	neha@abc.com
104	Ananya Rao	Mumbai	96660-44556	SQL	5000	Ravi Verma	ravi@abc.com
105	Vivaan Joshi	Delhi	95550-55667	Python	7000	Neha Gupta	neha@abc.com
102	Diya Mehta	Delhi	99880-22334	Excel, Python	3000, 7000	Neha Gupta	neha@abc.com

Two big design mistakes are hiding here:

- Duplicate data — Aarav's name, city and phone are stored again and again on every row.
- Multi-value columns — CourseName and Fees hold more than one value inside a single cell, e.g. "SQL, Python".

2. Issues With This Kind of Data

2.1 Data Redundancy (the same thing stored many times)

Aarav's phone number is written in 2 rows. Diya's in 2 rows. If the institute had 50 courses, one student's details could repeat 50 times.

- Wastes storage space.

- Makes the table unnecessarily large and slow.

2.2 Update Anomaly

Aarav changes his phone number. You must now update EVERY row that has his number.

If you update only one row, the database now says Aarav has two different phone numbers. Which one is correct? Nobody knows. This is called inconsistent data.

2.3 Insert Anomaly

A new trainer, Mr. Aslam, joins the institute but has not been assigned any student yet.

You cannot insert him — because this table needs a RollNo and a CourseName. You are forced to leave those blank or invent a fake student.

2.4 Delete Anomaly

Ananya (RollNo 104) cancels her admission, so you delete her row.

But that row was the only place holding some course/trainer information for that combination. Deleting the student accidentally deletes business data you still needed.

2.5 Multi-Value Columns Break Everything

The cell "SQL, Python" is not two values to a database. It is ONE single text string.

SQL has no idea there are two courses inside it. To SQL it is just a sentence.

2.6 No Reliable Unique Identifier

RollNo 101 appears twice, and 102 appears twice. So RollNo alone cannot uniquely identify a row. The table has no trustworthy key.

3. Challenges When Running Queries On This Data

3.1 Counting becomes wrong

Question: How many students do we have?

```
SELECT COUNT(*) FROM Student_Records;
```

Answer given: 7. Correct answer: 5. The query counted rows, not students.

You would have to remember to write:

```
SELECT COUNT(DISTINCT RollNo) FROM Student_Records;
```

Every single query now needs extra defensive logic. That is a burden.

3.2 Searching inside a multi-value column

Question: Show all students who took Python.

```
SELECT * FROM Student_Records  
WHERE CourseName = 'Python';
```

This returns only Vivaan. Aarav and Kabir are missed, because their cell says "SQL, Python", which is not equal to 'Python'.

So you are forced to use LIKE:

```
SELECT * FROM Student_Records  
WHERE CourseName LIKE '%Python%';
```

But LIKE is dangerous and slow:

- It cannot use an index, so the database scans the whole table.
- A course named 'Advanced Python' would also match, giving wrong results.
- A search for '%SQL%' would also match 'MySQL' and 'NoSQL'.

3.3 Maths on the Fees column is impossible

Question: What is the total fee collected?

```
SELECT SUM(Fees) FROM Student_Records;
```

This fails or returns garbage. Fees is text like "5000, 7000", not a number. SUM cannot add sentences.

3.4 Sorting and grouping give meaningless output

Grouping by CourseName creates fake groups such as 'SQL, Python' and 'Python, Excel' as if they were real courses.

```
SELECT CourseName, COUNT(*)  
FROM Student_Records  
GROUP BY CourseName;
```

You get a report full of categories that do not exist in reality.

3.5 Performance drops

- Bigger table because of repeated data means more disk reads.
- LIKE '%...%' searches cannot use indexes.
- Duplicates force DISTINCT everywhere, which is an expensive operation.

3.6 Data quality slowly rots

Someone types "Python " with a space, someone else types "python" in lowercase. Now the same course exists in three different spellings and no report is trustworthy.

4. What Can Be Done To Remove These Issues

The cure is called Normalization — the process of splitting one big messy table into smaller, focused tables.

Rule 1 — One cell must hold only one value (1NF)

Never store "SQL, Python" in a cell. Break it into separate rows.

Rule 2 — One table should describe one thing

- Student details belong in a Students table.
- Course details belong in a Courses table.
- Trainer details belong in a Trainers table.
- Who enrolled in what belongs in an Enrollments table.

Rule 3 — Every table needs a Primary Key

A column whose value is unique for every row, and never empty.

Rule 4 — Link tables using a Foreign Key

Instead of repeating the student's full name and phone everywhere, store just the RollNo. That small number points back to the Students table.

The Cleaned-Up Design

Table 1: Students

RollNo (PK)	StudentName	City	Phone
101	Aarav Sharma	Jaipur	98110-11223
102	Diya Mehta	Delhi	99880-22334
103	Kabir Singh	Jaipur	97770-33445
104	Ananya Rao	Mumbai	96660-44556
105	Vivaan Joshi	Delhi	95550-55667
106	Ishita Nair	Pune	94440-66778

Note: Ishita has joined but not enrolled in any course yet. The clean design allows this.

Table 2: Courses

CourseID (PK)	CourseName	Fees	TrainerID (FK)
C01	SQL	5000	T01
C02	Python	7000	T02
C03	Excel	3000	T02
C04	Power BI	6000	T03

Note: Power BI exists as a course even though nobody has enrolled in it yet.

Table 3: Trainers

TrainerID (PK)	TrainerName	Trainer_Email
T01	Ravi Verma	ravi@abc.com
T02	Neha Gupta	neha@abc.com

T03	Aslam Khan	aslam@abc.com
-----	------------	---------------

Note: Aslam is stored happily even with zero courses. The insert anomaly is gone.

Table 4: Enrollments (the bridge table)

EnrollID (PK)	RollNo (FK)	CourseID (FK)	EnrollDate
E1	101	C01	2026-01-10
E2	101	C02	2026-01-10
E3	101	C03	2026-02-05
E4	102	C01	2026-01-12
E5	102	C03	2026-03-01
E6	102	C02	2026-03-01
E7	103	C02	2026-01-20
E8	103	C03	2026-01-20
E9	104	C01	2026-02-14
E10	105	C02	2026-02-18

Notice what happened:

- Aarav's phone number is now stored exactly ONCE.
- Each cell holds a single value.
- Fees is a proper number, so SUM() works.
- A student with no course, and a course with no student, can both exist.

5. What Is The Issue After Splitting Into Separate Tables?

Splitting solved the storage problems, but it created a new practical problem.

5.1 The information is now scattered

Earlier, one simple SELECT * gave you everything. Now the answer to a simple question lives in four different places.

Question: Show student name, course name, fees and trainer name.

- StudentName is in Students.
- CourseName and Fees are in Courses.
- TrainerName is in Trainers.
- The connection between student and course is in Enrollments.

No single table can answer this question alone.

5.2 Codes are not human-readable

The Enrollments table says 101 took C02. That is meaningless to a manager reading a report. It must be translated back into "Aarav Sharma took Python".

5.3 More tables means more queries

Without a proper technique you would have to run one query per table and stitch the results together by hand in Excel. That is slow, manual and error-prone.

5.4 Referential integrity must be protected

What if someone enters CourseID 'C99' in Enrollments, but C99 does not exist in Courses? You now have an orphan record pointing to nothing.

The solution to all of this is the JOIN.

6. What Is a JOIN?

A JOIN is a SQL operation that combines rows from two or more tables based on a related column between them. In simple words: a JOIN glues the tables back together at query time — but only in the result, not in storage. You store data in a clean, split way. You read it in a combined, friendly way. Best of both worlds.

Simple example using our tables

Goal: show which student took which course.

```
SELECT s.StudentName,  
       c.CourseName,  
       c.Fees  
FROM Enrollments e  
JOIN Students s ON e.RollNo = s.RollNo  
JOIN Courses c ON e.CourseID = c.CourseID;
```

Reading it line by line:

- FROM Enrollments e — start from the bridge table, alias it as e.
- JOIN Students s ON e.RollNo = s.RollNo — for each enrollment, find the matching student.
- JOIN Courses c ON e.CourseID = c.CourseID — and find the matching course.
- SELECT — finally pick the columns we want to display.

Result:

StudentName	CourseName	Fees
Aarav Sharma	SQL	5000
Aarav Sharma	Python	7000
Aarav Sharma	Excel	3000
Diya Mehta	SQL	5000

Diya Mehta	Excel	3000
Diya Mehta	Python	7000
Kabir Singh	Python	7000
Kabir Singh	Excel	3000
Ananya Rao	SQL	5000
Vivaan Joshi	Python	7000

This looks like the original messy table — but every cell has one value, and nothing is duplicated in storage.

Adding the trainer as well

```
SELECT s.StudentName,  
       c.CourseName,  
       c.Fees,  
       t.TrainerName  
FROM Enrollments e  
JOIN Students s ON e.RollNo = s.RollNo  
JOIN Courses c ON e.CourseID = c.CourseID  
JOIN Trainers t ON c.TrainerID = t.TrainerID;
```

Three joins, four tables, one clean answer.

7. Why JOINS Become Important

7.1 They make normalization practical

Splitting tables is only a good idea if you can easily put them back together. JOINS are what make that possible. Without JOINS, normalization would be painful and nobody would do it.

7.2 They answer questions no single table can answer

Total fees collected per city, for example, needs Students (for city), Enrollments (for the link) and Courses (for fees).

```
SELECT s.City, SUM(c.Fees) AS Total_Fees  
FROM Enrollments e  
JOIN Students s ON e.RollNo = s.RollNo  
JOIN Courses c ON e.CourseID = c.CourseID  
GROUP BY s.City  
ORDER BY Total_Fees DESC;
```

Now SUM actually works, because Fees is a real number.

7.3 They turn codes into meaning

JOINS replace C02 with 'Python' and T01 with 'Ravi Verma' automatically, so reports are readable by humans.

7.4 They keep data accurate

Because each fact is stored in exactly one place, a JOIN always fetches the latest correct value. Update Aarav's phone once, and every report shows the new number instantly.

7.5 They are fast

JOINS work on keys, and keys are indexed. Matching an indexed number is far faster than searching for a substring with LIKE '%Python%'.

7.6 They let you find missing relationships

Certain JOIN types can reveal students with no course, or courses with no student — questions the old single table could never answer.

8. Primary Key and Foreign Key

8.1 Primary Key (PK)

A Primary Key is the column that uniquely identifies each row in a table.

Rules of a Primary Key:

- Its value must be unique — no two rows can share it.
- It can never be NULL (empty).
- It should never change once assigned.
- A table can have only one Primary Key (though it may span multiple columns).

Think of it like an Aadhaar number or a roll number — one person, one number, forever.

In our design:

- Students → RollNo
- Courses → CourseID
- Trainers → TrainerID
- Enrollments → EnrollID

8.2 Foreign Key (FK)

A Foreign Key is a column in one table that points to the Primary Key of another table.

It is the actual link, the rope tying two tables together.

Rules of a Foreign Key:

- Its value must already exist in the parent table's Primary Key.
- It can repeat — Aarav's RollNo 101 appears three times in Enrollments, and that is perfectly fine.
- It may be NULL if the relationship is optional.

In our design:

- Enrollments.RollNo → points to Students.RollNo
- Enrollments.CourseID → points to Courses.CourseID
- Courses.TrainerID → points to Trainers.TrainerID

8.3 Side-by-side comparison

Point	Primary Key	Foreign Key
Purpose	Identifies a row uniquely	Links to another table
Duplicates	Not allowed	Allowed
NULL values	Never allowed	Allowed if optional
How many per table	Only one	Can have many
Where it lives	In its own table	In the child table
Example	Students.RollNo	Enrollments.RollNo

8.4 Creating them in SQL

```

CREATE TABLE Students (
  RollNo INT PRIMARY KEY,
  StudentName VARCHAR(50) NOT NULL,
  City VARCHAR(30),
  Phone VARCHAR(15)
);

CREATE TABLE Trainers (
  TrainerID VARCHAR(5) PRIMARY KEY,
  TrainerName VARCHAR(50),
  Trainer_Email VARCHAR(50)
);

CREATE TABLE Courses (
  CourseID VARCHAR(5) PRIMARY KEY,
  CourseName VARCHAR(50),
  Fees DECIMAL(10,2),
  TrainerID VARCHAR(5),
  FOREIGN KEY (TrainerID) REFERENCES Trainers(TrainerID)
);

CREATE TABLE Enrollments (
  EnrollID VARCHAR(5) PRIMARY KEY,
  RollNo INT,
  CourseID VARCHAR(5),
  EnrollDate DATE,
  FOREIGN KEY (RollNo) REFERENCES Students(RollNo),
  FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)
);

```

8.5 Why the Foreign Key protects you

Once the FOREIGN KEY is declared, the database itself refuses bad data:

```

INSERT INTO Enrollments
VALUES ('E11', 999, 'C01', '2026-04-01');

```

```
-- ERROR: RollNo 999 does not exist in Students
```

This is called Referential Integrity. The database becomes your safety net.

9. How JOINS Solve The Original Problem

Original Problem	How the new design + JOIN fixes it
Duplicate student details	Student stored once in Students; JOIN fetches it when needed
Multi-value cell 'SQL, Python'	Each course is a separate row in Enrollments
SUM(Fees) failed	Fees is now a proper number column in Courses
COUNT(*) gave wrong student count	COUNT(*) on Students gives the exact truth
Update anomaly	Change the phone in one row; JOIN shows it everywhere
Insert anomaly	Trainer Aslam and course Power BI exist with no students
Delete anomaly	Deleting an enrollment does not delete the course or student
Slow LIKE '%Python%' search	Exact match on indexed CourseID — fast and accurate
Cannot find students with no course	LEFT JOIN reveals them instantly

Before and after, in one query

Old way (wrong result):

```
SELECT COUNT(*) FROM Student_Records
WHERE CourseName LIKE '%Python%'; -- unreliable
```

New way (correct result):

```
SELECT COUNT(DISTINCT e.RollNo) AS Python_Students
FROM Enrollments e
JOIN Courses c ON e.CourseID = c.CourseID
WHERE c.CourseName = 'Python'; -- exact, indexed, correct
```

10. Types of JOINS in SQL

For the examples below we mainly use Students and Enrollments, because Ishita (106) has no enrollment — which makes the differences visible.

10.1 INNER JOIN

Returns only the rows that have a match in BOTH tables.

If a student has no enrollment, that student disappears from the result.

```
SELECT s.StudentName, c.CourseName
FROM Students s
INNER JOIN Enrollments e ON s.RollNo = e.RollNo
INNER JOIN Courses c ON e.CourseID = c.CourseID;
```

Result: 10 rows. Ishita is NOT shown, because she has no enrollment.

Use it when: you want only complete, matched information.

10.2 LEFT JOIN (LEFT OUTER JOIN)

Returns ALL rows from the left table, plus matching rows from the right table.

Where there is no match, the right side columns are filled with NULL.

```
SELECT s.StudentName, c.CourseName
FROM Students s
LEFT JOIN Enrollments e ON s.RollNo = e.RollNo
LEFT JOIN Courses c ON e.CourseID = c.CourseID;
```

Result (last rows shown):

StudentName	CourseName
Ananya Rao	SQL
Vivaan Joshi	Python
Ishita Nair	NULL

A very common real use — find students who enrolled in nothing:

```
SELECT s.StudentName
FROM Students s
LEFT JOIN Enrollments e ON s.RollNo = e.RollNo
WHERE e.RollNo IS NULL; -- returns Ishita Nair
```

Use it when: you want everything from the main table, matched or not.

10.3 RIGHT JOIN (RIGHT OUTER JOIN)

The mirror image of LEFT JOIN. Returns ALL rows from the right table, plus matches from the left.

```
SELECT s.StudentName, c.CourseName
FROM Enrollments e
RIGHT JOIN Courses c ON e.CourseID = c.CourseID
LEFT JOIN Students s ON e.RollNo = s.RollNo;
```

Result includes Power BI with NULL student, because nobody enrolled in it.

Use it when: you want everything from the second table. Note that most developers simply flip the table order and use LEFT JOIN instead, because it reads more naturally.

10.4 FULL OUTER JOIN

Returns ALL rows from BOTH tables. Unmatched sides are filled with NULL.

```
SELECT s.StudentName, c.CourseName
FROM Students s
FULL OUTER JOIN Enrollments e ON s.RollNo = e.RollNo
FULL OUTER JOIN Courses c ON e.CourseID = c.CourseID;
```

This shows Ishita (student with no course) AND Power BI (course with no student) in the same result.

Note: MySQL does not support FULL OUTER JOIN directly. Simulate it with a LEFT JOIN, then UNION, then a RIGHT JOIN.

10.5 CROSS JOIN

Returns every possible combination of rows from both tables. No ON condition is used.

6 students × 4 courses = 24 rows.

```
SELECT s.StudentName, c.CourseName
FROM Students s
CROSS JOIN Courses c;
```

Use it when: you deliberately need all combinations, for example building a blank attendance grid or a price matrix.

Warning: forgetting the ON clause in a normal join accidentally creates a CROSS JOIN. On large tables this can produce millions of rows and crash your query.

10.6 SELF JOIN

A table joined to itself. Useful when rows in a table relate to other rows in the same table.

Example: find pairs of students living in the same city.

```
SELECT a.StudentName AS Student_1,
       b.StudentName AS Student_2,
       a.City
FROM Students a
JOIN Students b
ON a.City = b.City
AND a.RollNo < b.RollNo;
```

Result:

Student_1	Student_2	City
Aarav Sharma	Kabir Singh	Jaipur
Diya Mehta	Vivaan Joshi	Delhi

The condition a.RollNo < b.RollNo prevents a student pairing with himself and stops the same pair appearing twice.

10.7 Quick comparison of all JOIN types

JOIN Type	What it returns	Ishita (no course)?	Power BI (no student)?
INNER JOIN	Only matching rows	No	No
LEFT JOIN	All left + matches	Yes	No
RIGHT JOIN	All right + matches	No	Yes
FULL OUTER JOIN	Everything from both	Yes	Yes
CROSS JOIN	Every combination	Yes (paired with all)	Yes (paired with all)
SELF JOIN	Table matched to itself	Depends on condition	Not applicable

11. Practice Queries To Try

Try writing these yourself before looking at the hints.

1. Show every student along with the total fees they have to pay.

- Hint: Join Students, Enrollments, Courses; use SUM with GROUP BY.

2. List courses that nobody has enrolled in.

- Hint: LEFT JOIN from Courses to Enrollments, then check IS NULL.

3. Find how many students each trainer teaches.

- Hint: Join Trainers, Courses, Enrollments; COUNT DISTINCT RollNo.

4. Show all students from Jaipur and their courses.

- Hint: Add WHERE s.City = 'Jaipur' to the three-table join.

5. Find the course with the highest number of enrollments.

- Hint: GROUP BY CourseName, ORDER BY COUNT(*) DESC, LIMIT 1.

12. Key Takeaways

- Never store more than one value inside a single cell.
- Never repeat the same information across many rows.
- Split data into focused tables — one table describes one thing.
- Give every table a Primary Key.
- Link tables using Foreign Keys to protect data integrity.
- Use JOINS to bring the pieces back together whenever you need a full picture.
- Choose INNER JOIN for matched data, LEFT JOIN when you also need the unmatched rows.